Name : Pranav Soitkar

Class : C2

Batch : B2

Roll no. : 24

LAB ML 8

Aim: To implement Support Vector Machine (SVM) algorithm for classification.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.svm import SVC

from sklearn.metrics import (accuracy_score,precision_score, recall_score, f1_score

import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
df = pd.read_csv("ai4i2020.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	T
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	

In [4]:
```
df.tail()
```

Out[4]:

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machi failu
9995	9996	M24855	M	298.8	308.4	1604	29.5	14	
9996	9997	H39410	H	298.9	308.4	1632	31.8	17	
9997	9998	M24857	M	299.0	308.6	1645	33.4	22	
9998	9999	H39412	H	299.0	308.7	1408	48.5	25	
9999	10000	M24859	M	299.0	308.7	1500	40.2	30	

In [5]:
```
df.shape
```

Out[5]:
```
(10000, 14)
```

In [6]:
```
df.columns
```

Out[6]:
```
Index(['UDI', 'Product ID', 'Type', 'Air temperature [K]',
       'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]',
       'Tool wear [min]', 'Machine failure', 'TWF', 'HDF', 'PWF', 'OSF',
       'RNF'],
      dtype='str')
```

In [7]:
```
df.dtypes
```

Out[7]:
```
UDI                        int64
Product ID                   str
Type                         str
Air temperature [K]        float64
Process temperature [K]    float64
Rotational speed [rpm]       int64
Torque [Nm]                float64
Tool wear [min]              int64
Machine failure              int64
TWF                          int64
HDF                          int64
PWF                          int64
OSF                          int64
RNF                          int64
dtype: object
```

In [8]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   UDI                      10000 non-null  int64
 1   Product ID               10000 non-null  str
 2   Type                     10000 non-null  str
 3   Air temperature [K]      10000 non-null  float64
 4   Process temperature [K]  10000 non-null  float64
 5   Rotational speed [rpm]   10000 non-null  int64
 6   Torque [Nm]              10000 non-null  float64
 7   Tool wear [min]          10000 non-null  int64
 8   Machine failure          10000 non-null  int64
 9   TWF                      10000 non-null  int64
 10  HDF                      10000 non-null  int64
 11  PWF                      10000 non-null  int64
 12  OSF                      10000 non-null  int64
 13  RNF                      10000 non-null  int64
dtypes: float64(3), int64(9), str(2)
memory usage: 1.1 MB
```

In [9]:
```
df.describe()
```

Out[9]:

	UDI	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	5000.50000	300.004930	310.005560	1538.776100	39.986910	107.951000
std	2886.89568	2.000259	1.483734	179.284096	9.968934	63.654147
min	1.00000	295.300000	305.700000	1168.000000	3.800000	0.000000
25%	2500.75000	298.300000	308.800000	1423.000000	33.200000	53.000000
50%	5000.50000	300.100000	310.100000	1503.000000	40.100000	108.000000
75%	7500.25000	301.500000	311.100000	1612.000000	46.800000	162.000000
max	10000.00000	304.500000	313.800000	2886.000000	76.600000	253.000000

In [10]:
```
df.describe().T
```

Out[10]:

	count	mean	std	min	25%	50%	75%	max
UDI	10000.0	5000.50000	2886.895680	1.0	2500.75	5000.5	7500.25	10000.0
Air temperature [K]	10000.0	300.00493	2.000259	295.3	298.30	300.1	301.50	304.5
Process temperature [K]	10000.0	310.00556	1.483734	305.7	308.80	310.1	311.10	313.8
Rotational speed [rpm]	10000.0	1538.77610	179.284096	1168.0	1423.00	1503.0	1612.00	2886.0
Torque [Nm]	10000.0	39.98691	9.968934	3.8	33.20	40.1	46.80	76.6
Tool wear [min]	10000.0	107.95100	63.654147	0.0	53.00	108.0	162.00	253.0
Machine failure	10000.0	0.03390	0.180981	0.0	0.00	0.0	0.00	1.0
TWF	10000.0	0.00460	0.067671	0.0	0.00	0.0	0.00	1.0
HDF	10000.0	0.01150	0.106625	0.0	0.00	0.0	0.00	1.0
PWF	10000.0	0.00950	0.097009	0.0	0.00	0.0	0.00	1.0
OSF	10000.0	0.00980	0.098514	0.0	0.00	0.0	0.00	1.0
RNF	10000.0	0.00190	0.043550	0.0	0.00	0.0	0.00	1.0

In [11]:
```
df.isnull()
```

Out[11]:

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
9995	False	False	False	False	False	False	False	False	False
9996	False	False	False	False	False	False	False	False	False
9997	False	False	False	False	False	False	False	False	False
9998	False	False	False	False	False	False	False	False	False
9999	False	False	False	False	False	False	False	False	False

10000 rows × 14 columns

In [12]:
```
df.duplicated().sum()
```

Out[12]:
```
np.int64(0)
```

In [13]:
```
df.duplicated().sum()
```

Out[13]:
```
np.int64(0)
```

In [14]:
```
df["Type"].unique()
```

Out[14]:
```
<ArrowStringArray>
['M', 'L', 'H']
Length: 3, dtype: str
```

In [15]:
```
df["Machine failure"].value_counts()
```

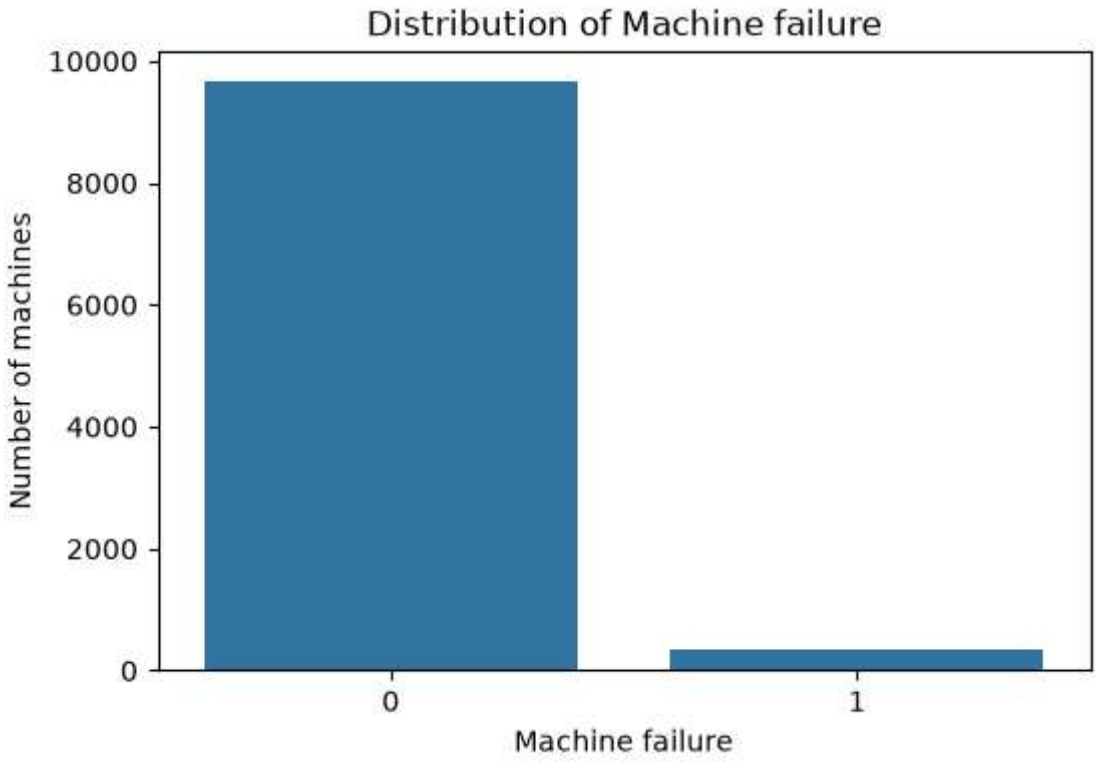
Out[15]:
```
Machine failure
0    9661
1     339
Name: count, dtype: int64
```

In [16]:
```
plt.figure(figsize=(6,4))

sns.countplot(x="Machine failure", data=df)

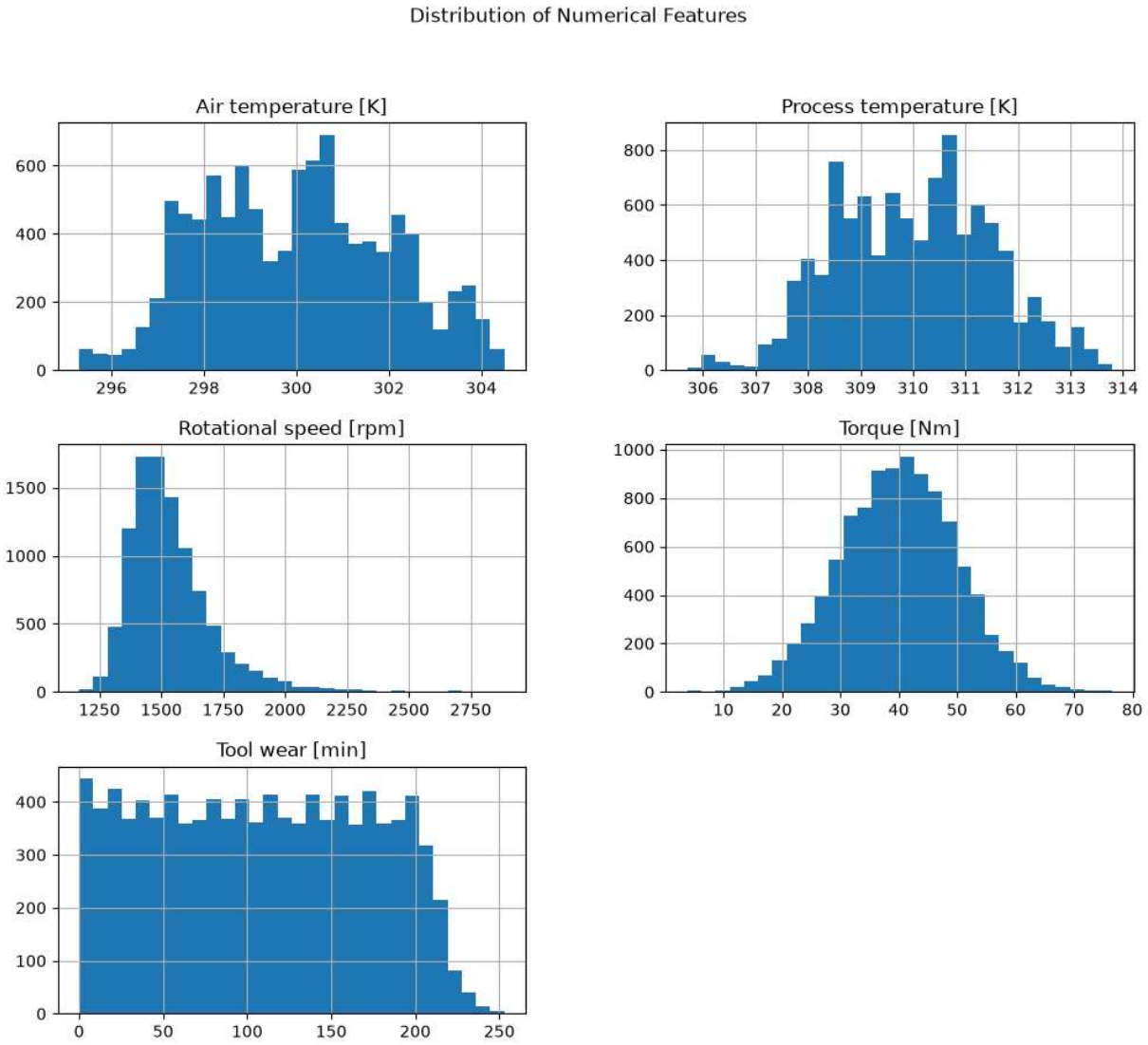
plt.title("Distribution of Machine failure")
plt.xlabel("Machine failure")
```

```
plt.ylabel("Number of machines")
plt.show()
```



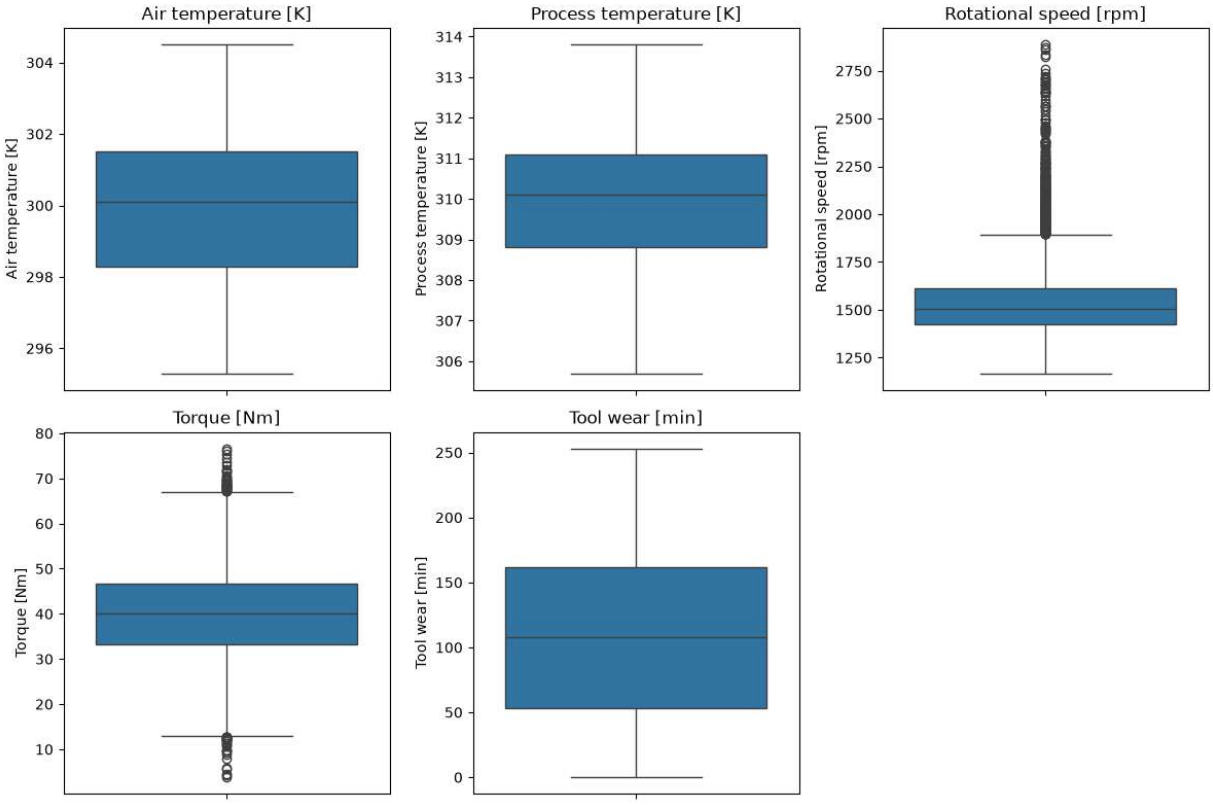
In [17]:
```
numerical_features = ["Air temperature [K]", "Process temperature [K]", "Rotational
```

In [18]:
```
df[numerical_features].hist(figsize=(12,10), bins = 30)
plt.suptitle("Distribution of Numerical Features")
plt.show()
```



In [19]:
```
plt.figure(figsize=(12,8))

for i, column in enumerate(numerical_features, 1):
    plt.subplot(2,3,i)
    sns.boxplot(y=df[column])
    plt.title(column)
plt.tight_layout()
plt.show()
```

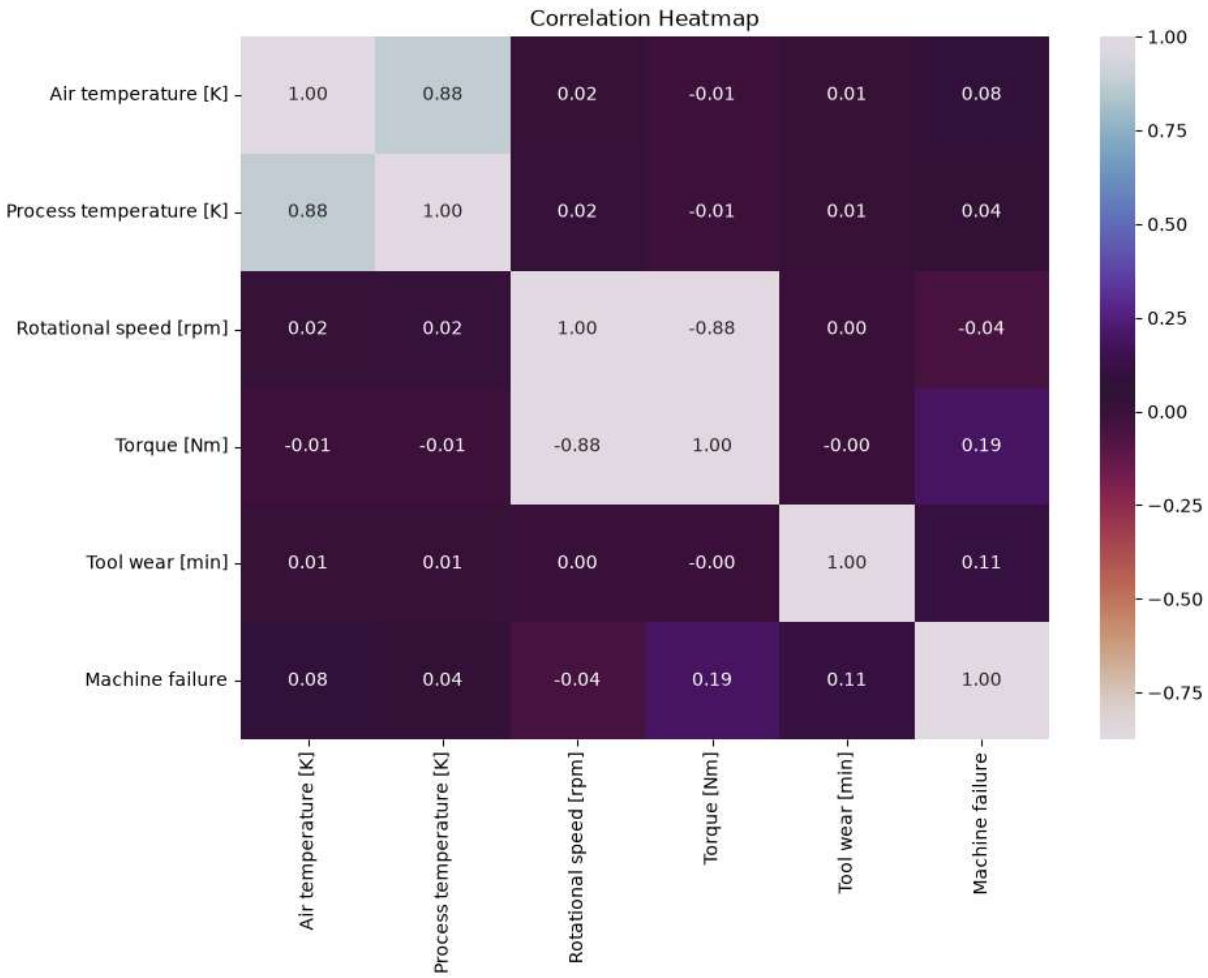


In [20]:

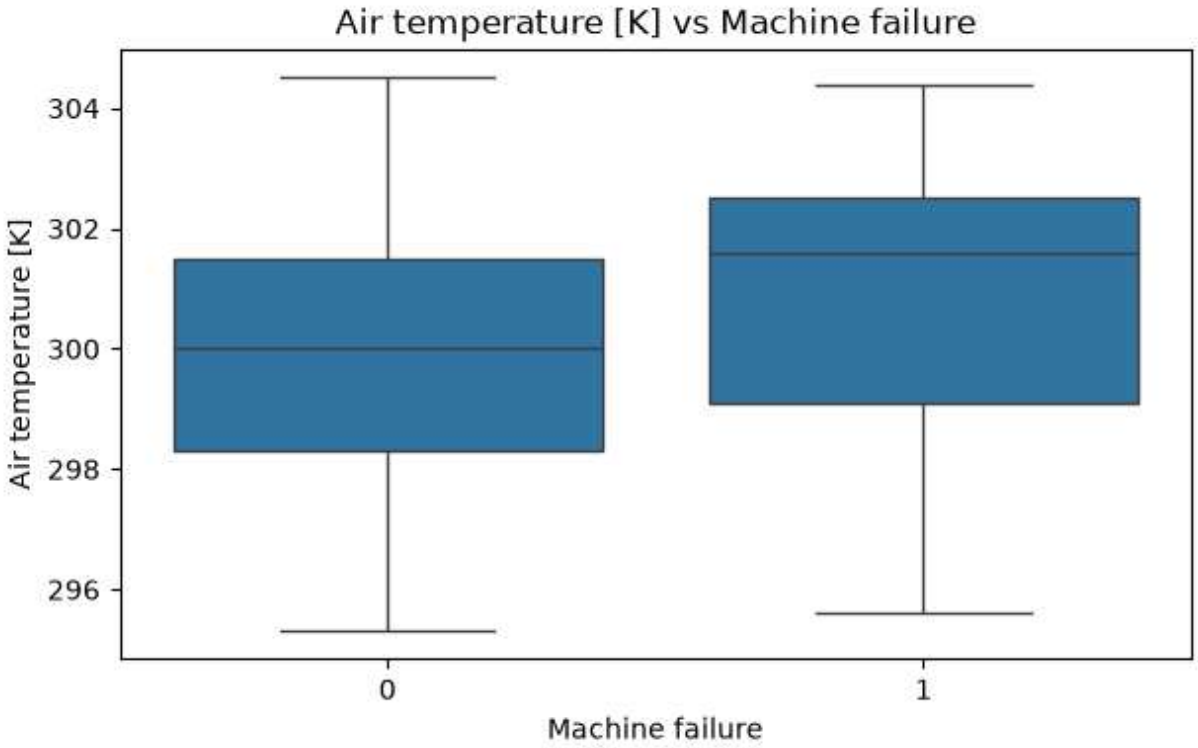
```
plt.figure(figsize=(10,7))

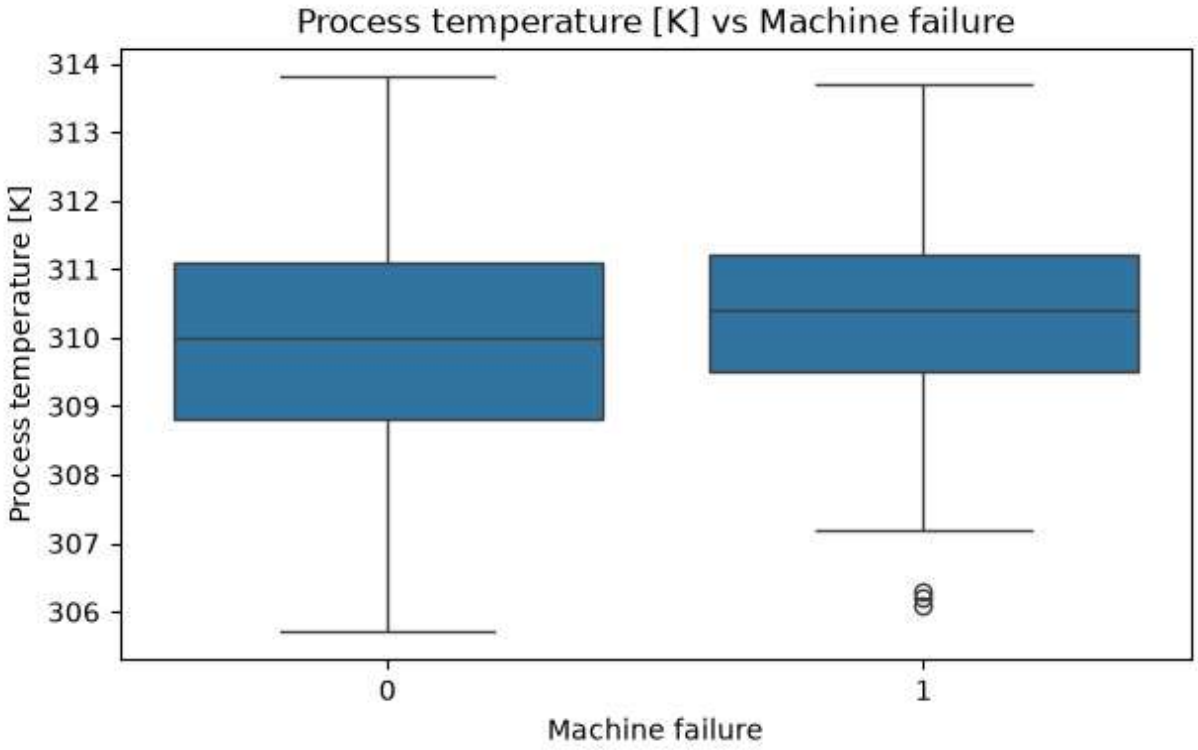
correlation = df[numerical_features + ["Machine failure"]].corr()

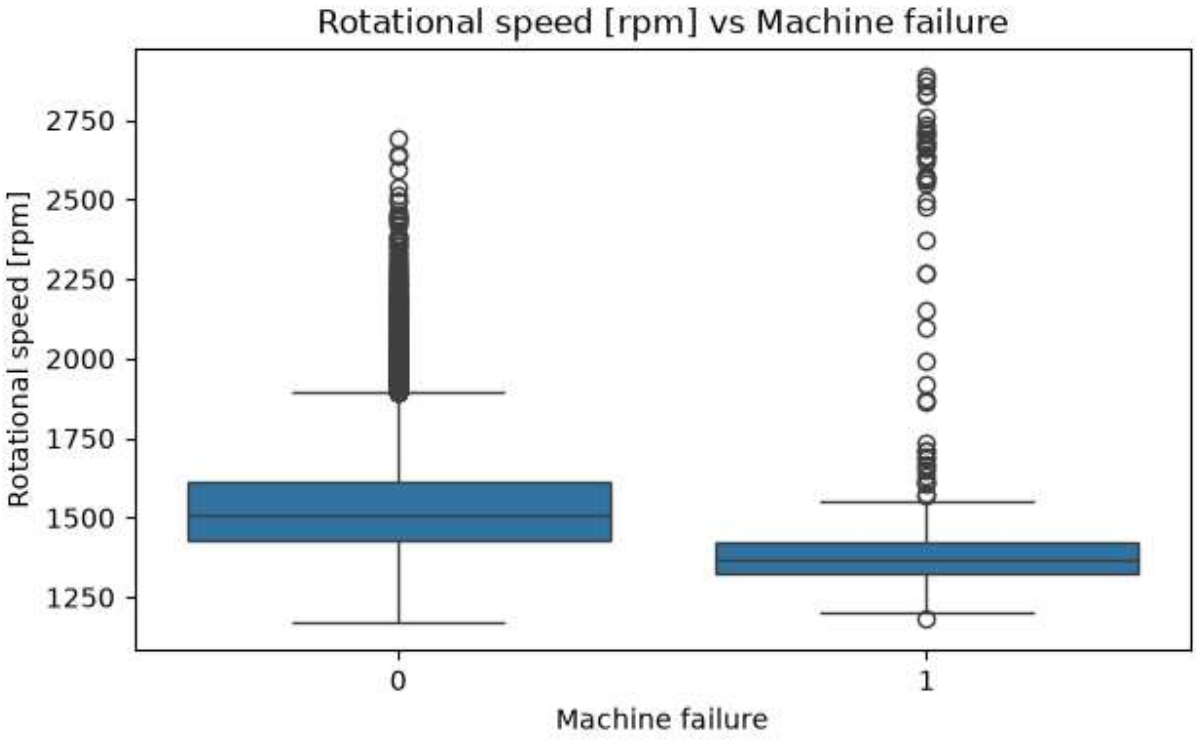
sns.heatmap(correlation, annot=True, cmap="twilight_r",fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

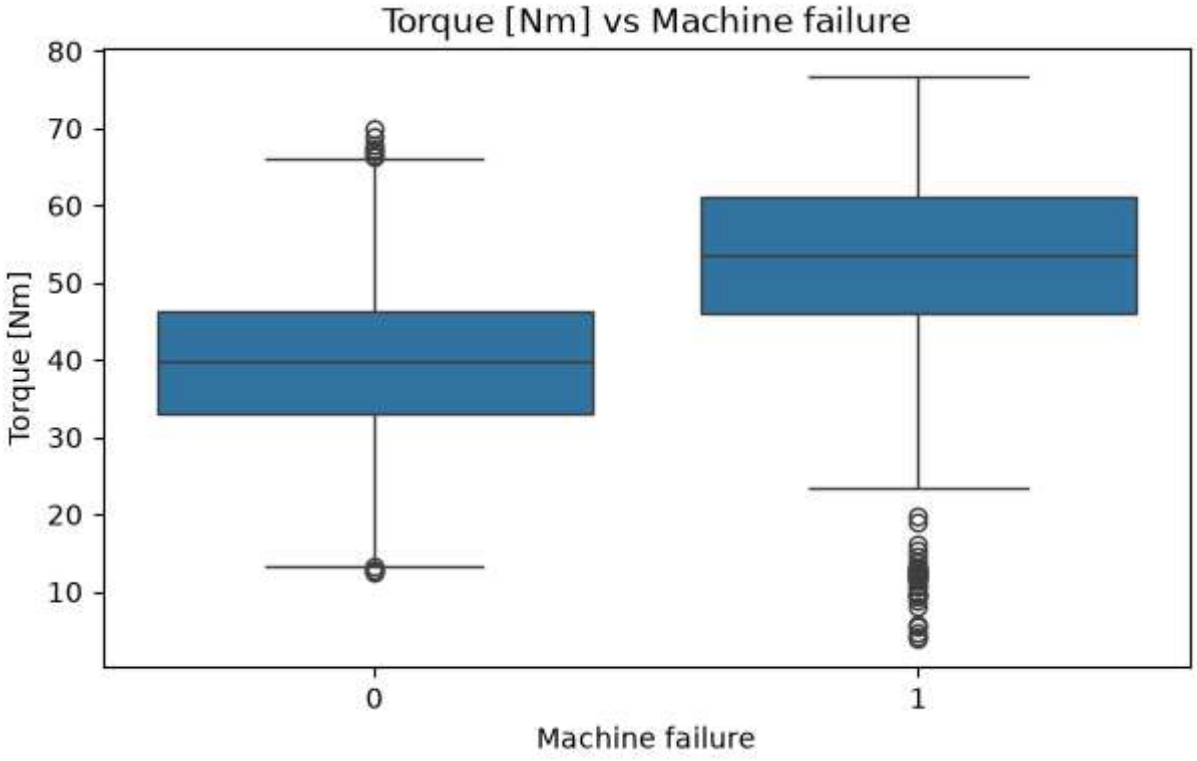


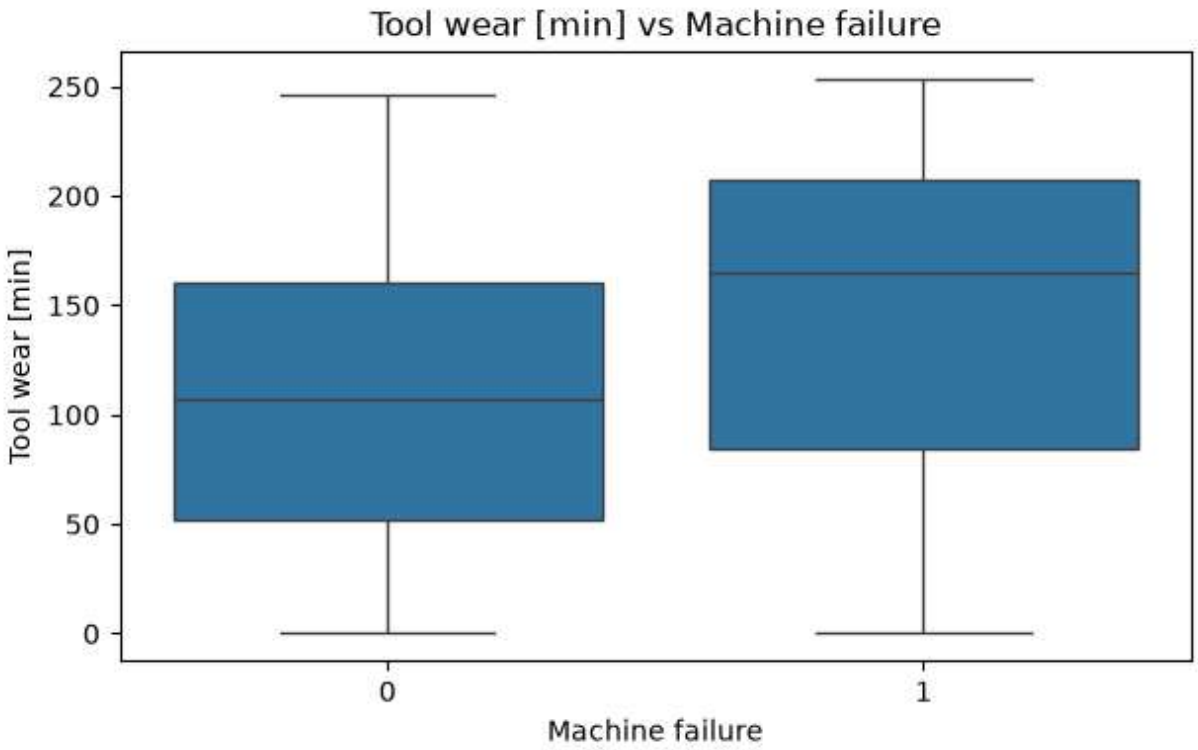
In [21]:
```
for column in numerical_features:
  plt.figure(figsize=(7,4))
  sns.boxplot(x="Machine failure", y=column, data=df)
  plt.title(f"{column} vs Machine failure")
plt.show()
```


Air temperature [K] vs Machine failure
Air temperature [K]
304
302
300
298
296
0
1
Machine failure

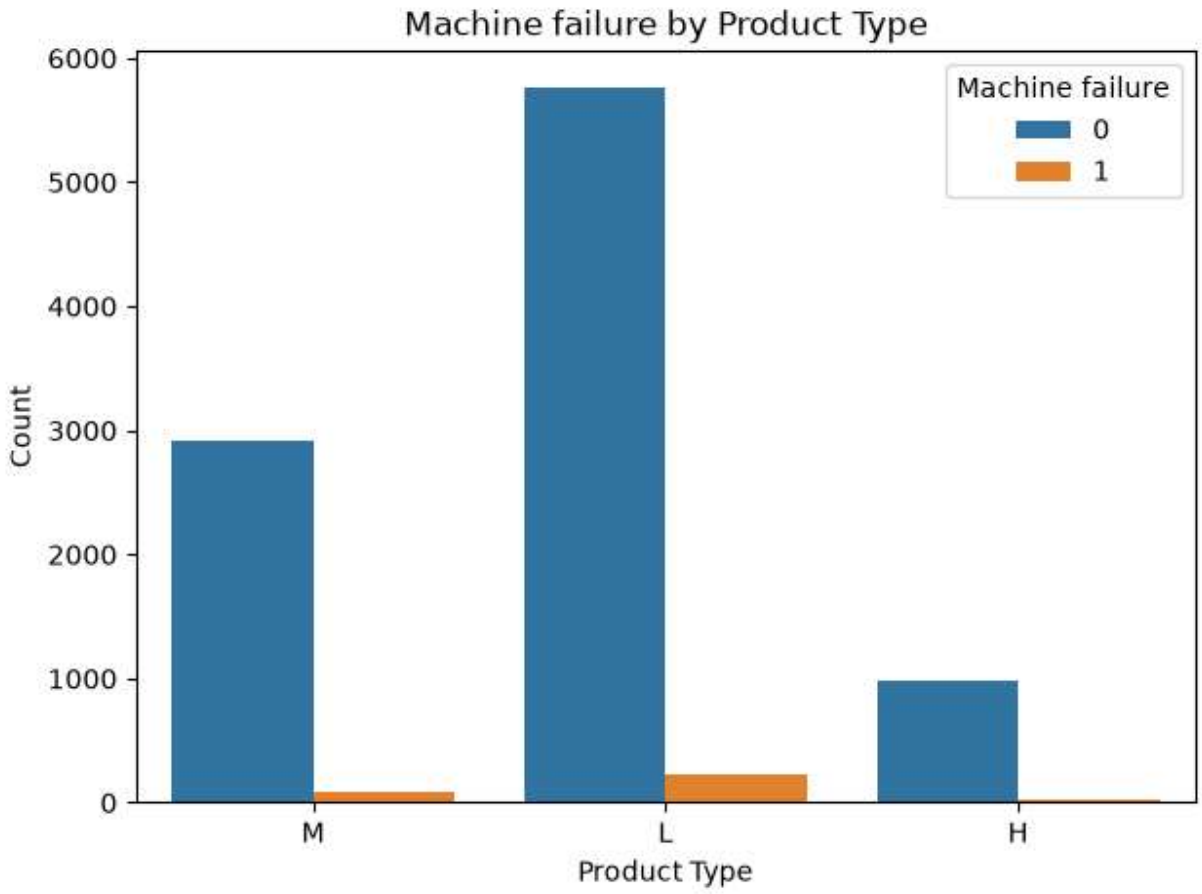

Process temperature [K] vs Machine failure
Process temperature [K]
314
313
312
311
310
309
308
307
306
0
1
Machine failure


Rotational speed [rpm] vs Machine failure
Rotational speed [rpm]
2750
2500
2250
2000
1750
1500
1250
0
1
Machine failure


Torque [Nm] vs Machine failure
Torque [Nm]
80
70
60
50
40
30
20
10
0
1
Machine failure



In [22]:
```
plt.figure(figsize=(7,5))
sns.countplot(x="Type", hue="Machine failure", data=df)
plt.title("Machine failure by Product Type")
plt.xlabel("Product Type")
plt.ylabel("Count")
plt.show()
```



In [23]:
```
X = df[
    [
        "Type",
        "Air temperature [K]",
        "Process temperature [K]",
        "Rotational speed [rpm]",
        "Torque [Nm]",
        "Tool wear [min]",
    ]
]
```

In [24]:
```
y = df["Machine failure"]
```

In [25]:
```
print("Input shape :", X.shape)
print("Output shape :", y.shape)
```

```
Input shape : (10000, 6)
Output shape : (10000,)
```

In [26]:
```
categorical_features = ["Type"]

numerical_features = [
    "Air temperature [K]",
    "Process temperature [K]",
    "Rotational speed [rpm]",
    "Torque [Nm]",
```

```
    "Tool wear [min]",
]
```

In [27]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

print("Training Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])
```

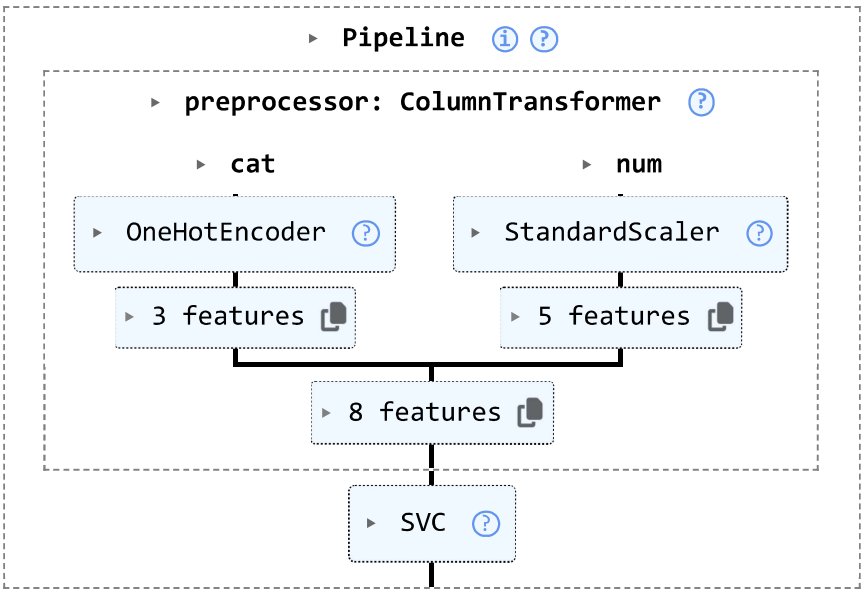
```
Training Samples: 8000
Testing Samples: 2000
```

In [28]:
```
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(), categorical_features),
        ("num", StandardScaler(), numerical_features),
    ]
)
```

In [29]:
```
linear_svm = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", SVC(kernel="linear")),
    ]
)
```

In [30]:
```
linear_svm.fit(X_train, y_train)
```

Out[30]:



In [31]:
```
y_pred_linear = linear_svm.predict(X_test)
```

In [32]:
```
accuracy_linear = accuracy_score(y_test, y_pred_linear)

precision_linear = precision_score(y_test, y_pred_linear)

recall_linear = recall_score(y_test, y_pred_linear)

f1_linear = f1_score(y_test, y_pred_linear)
```

```
print("Accuracy :", accuracy_linear)
print("Precision :", precision_linear)
print("Recall :", recall_linear)
print("F1 Score :", f1_linear)
```

```
Accuracy : 0.966
Precision : 0.0
Recall : 0.0
F1 Score : 0.0
```

In [33]:
```
print(
    classification_report(
        y_test,
        y_pred_linear,
        target_names = ["No Failure", "Failure"],
        zero_division = 0
    )
)
```

```
              precision    recall  f1-score   support

  No Failure       0.97      1.00      0.98      1932
     Failure       0.00      0.00      0.00        68

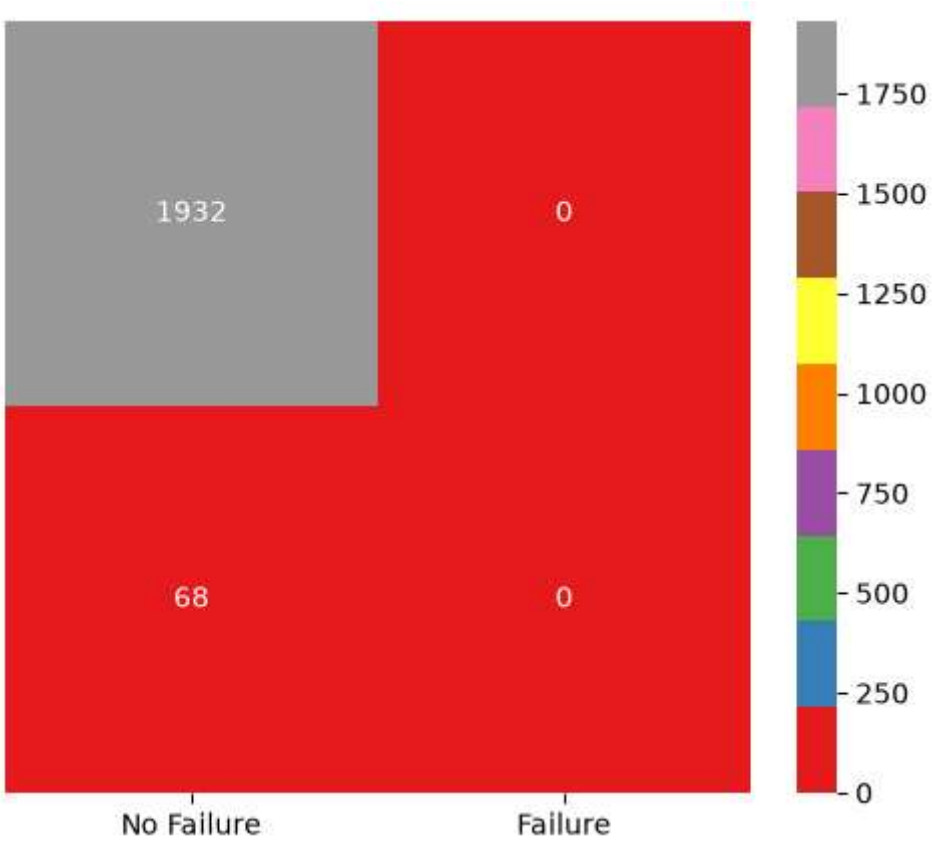
    accuracy                           0.97      2000
   macro avg       0.48      0.50      0.49      2000
weighted avg       0.93      0.97      0.95      2000
```

In [34]:
```
cm_linear = confusion_matrix(y_test, y_pred_linear)

plt.figure(figsize=(6,5))

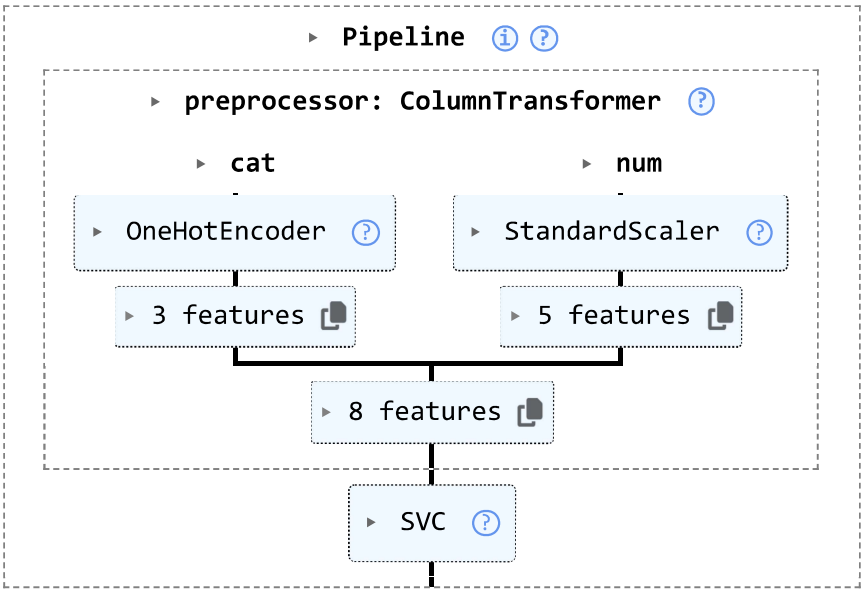
sns.heatmap(cm_linear, annot = True, fmt="d", cmap="Set1",
            xticklabels=["No Failure","Failure"],
            yticklabels=[]
            )
```

Out[34]: <Axes: >



In [37]:
```
rbf_svm = Pipeline(
    steps=[("preprocessor", preprocessor),
     ("classifier", SVC(kernel = "rbf"))])
rbf_svm.fit(X_train, y_train)
```

Out[37]:



In [38]:
```
y_pred_rbf = rbf_svm.predict(X_test)
```

In [39]:
```
accuracy_rbf = accuracy_score(y_test, y_pred_rbf)

precision_rbf = precision_score(y_test, y_pred_rbf)
```

```
recall_rbf = recall_score(y_test, y_pred_rbf)

f1_rbf = f1_score(y_test, y_pred_rbf)

print("RBF SVM Results")
print("Accuracy :", accuracy_rbf)
print("Precision :", precision_rbf)
print("Recall :", recall_rbf)
print("F1 Score :", f1_rbf)
```

```
RBF SVM Results
Accuracy : 0.972
Precision : 0.875
Recall : 0.20588235294117646
F1 Score : 0.3333333333333333
```

In [40]:
```
print(classification_report(y_test, y_pred_rbf, target_names = ["No Failure", "Fail
```

```
              precision    recall  f1-score   support

  No Failure       0.97      1.00      0.99      1932
     Failure       0.88      0.21      0.33        68

    accuracy                           0.97      2000
   macro avg       0.92      0.60      0.66      2000
weighted avg       0.97      0.97      0.96      2000
```

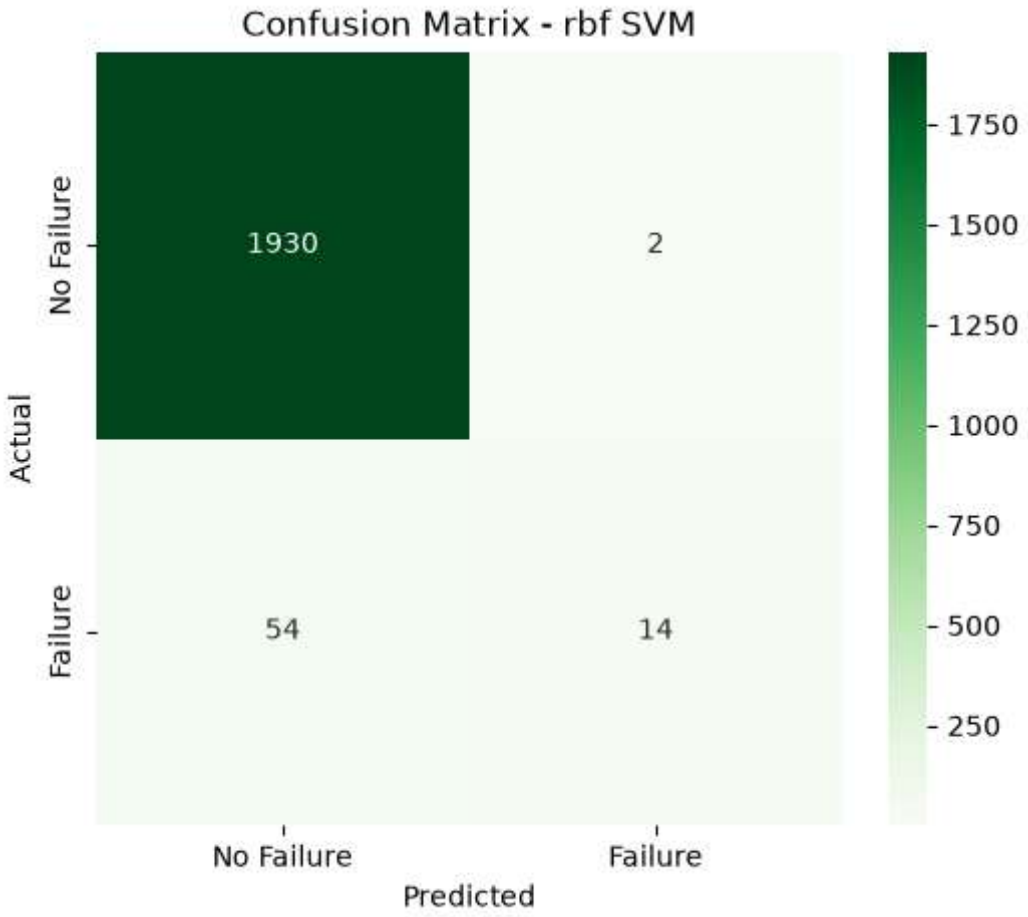
In [41]:
```
cm_rbf = confusion_matrix(y_test, y_pred_rbf)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_rbf,
    annot = True,
    fmt = "d",
    cmap = "Greens",
    xticklabels = ["No Failure", "Failure"],
    yticklabels = ["No Failure", "Failure"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - rbf SVM")

plt.show()
```



In [42]:
```
comparison = pd.DataFrame({
    "Model": ["Linear SVM", "RBF SVM"],
    "Accuracy": [accuracy_linear, accuracy_rbf],
    "Precision": [precision_linear, precision_rbf],
    "Recall": [recall_linear, recall_rbf],
    "F1 Score": [f1_linear, f1_rbf]
})

comparison
```

Out[42]:

	Model	Accuracy	Precision	Recall	F1 Score
0	Linear SVM	0.966	0.000	0.000000	0.000000
1	RBF SVM	0.972	0.875	0.205882	0.333333

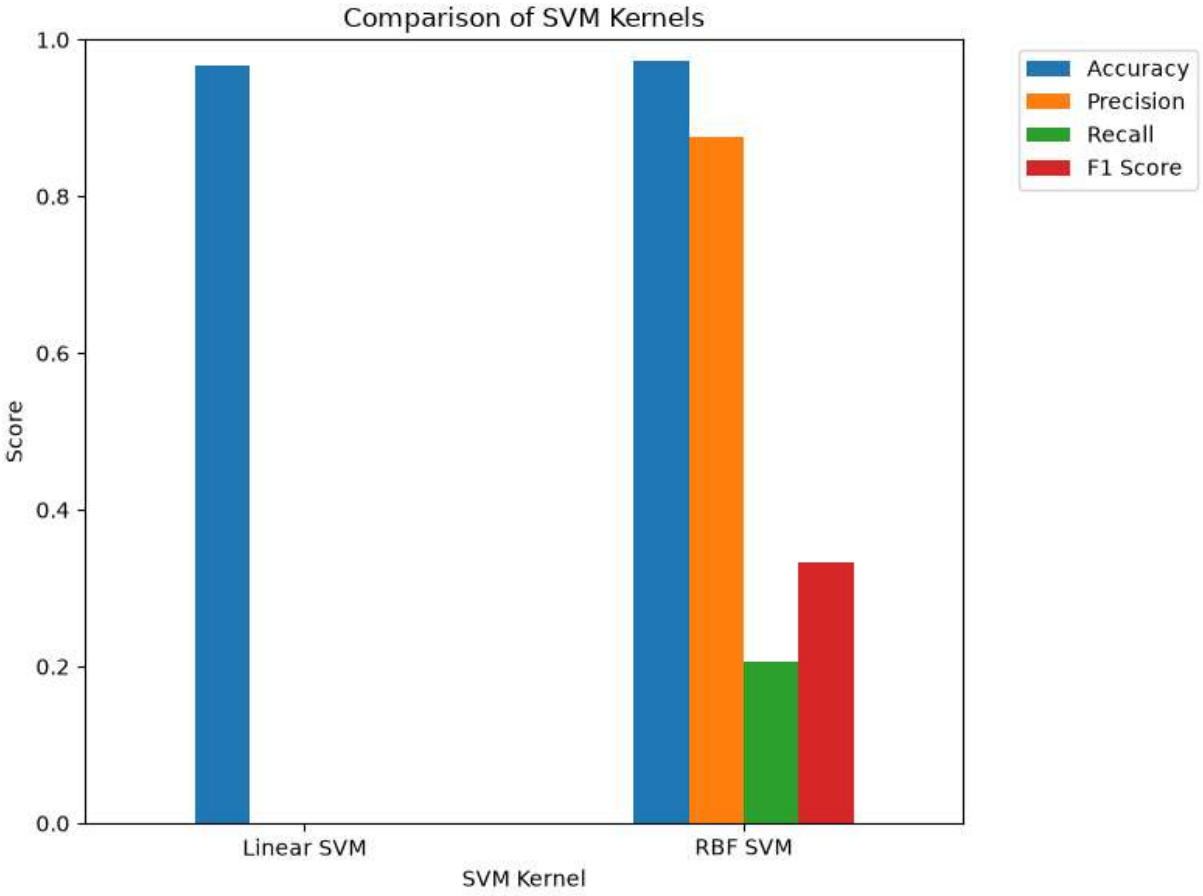
In [44]:
```
comparison_plot = comparison.set_index("Model")

comparison_plot.plot(
    kind = "bar",
    figsize = (8, 6),
)
plt.title("Comparison of SVM Kernels")
plt.xlabel("SVM Kernel")
plt.ylabel("Score")
plt.ylim(0, 1)
```

```
plt.xticks(rotation = 0)

plt.legend(bbox_to_anchor = (1.05, 1), loc = "upper left")

plt.tight_layout()
plt.show()
```



In [45]:
```
from sklearn.pipeline import Pipeline
pipeline = Pipeline(
    steps = [("preprocessor", preprocessor), ("classifier", SVC())]
)
```

In [47]:
```
param_grid = [{"classifier__kernel": ["linear"], "classifier__C":[0.1, 1, 10, 100]}
              {"classifier__kernel":["rbf"],
               "classifier__C": [0.1, 1, 10, 100], "classifier__gamma":["scale", 0.
```

In [48]:
```
grid_search = GridSearchCV(estimator = pipeline, param_grid = param_grid, cv = 5, s
```

In [49]:
```
grid_search.fit(X_train, y_train)

print("GridSearch CV completed")
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
GridSearch CV completed
```

In [50]:
```
print("Best Parameter")
```

```
print(grid_search.best_params_)
```

```
Best Parameter
{'classifier__C': 100, 'classifier__gamma': 'scale', 'classifier__kernel': 'rbf'}
```

In [51]:
```
print("Best Cross-Validation F1-score: ", grid_search.best_score_)
```

```
Best Cross-Validation F1-score:  0.7080372186703207
```

In [52]:
```
best_model = grid_search.best_estimator_

y_pred_tuned = best_model.predict(X_test)
```

In [53]:
```
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)

precision_tuned = precision_score(y_test, y_pred_tuned)

recall_tuned = recall_score(y_test, y_pred_tuned)

f1_tuned = f1_score(y_test, y_pred_tuned)

print("Tuned SVM Results")
print("Accuracy :", accuracy_tuned)
print("Precision :", precision_tuned)
print("Recall :", recall_tuned)
print("F1 Score :", f1_tuned)
```

```
Tuned SVM Results
Accuracy : 0.98
Precision : 0.7592592592592593
Recall : 0.6029411764705882
F1 Score : 0.6721311475409836
```

In [55]:
```
final_comparison = pd.DataFrame({
    "Model": ["Linear SVM", "RBF SVM", "Tuned SVM"],
    "Accuracy": [accuracy_linear, accuracy_rbf, accuracy_tuned],
    "Precision": [precision_linear, precision_rbf, precision_tuned],
    "Recall ": [recall_linear, recall_rbf, recall_tuned],
    "F1 Score": [f1_linear, f1_rbf, f1_tuned]
})

final_comparison
```

Out[55]:

	Model	Accuracy	Precision	Recall	F1 Score
0	Linear SVM	0.966	0.000000	0.000000	0.000000
1	RBF SVM	0.972	0.875000	0.205882	0.333333
2	Tuned SVM	0.980	0.759259	0.602941	0.672131

In [56]:
```
final_comparison.set_index("Model").plot(kind = "bar", figsize = (11, 6))

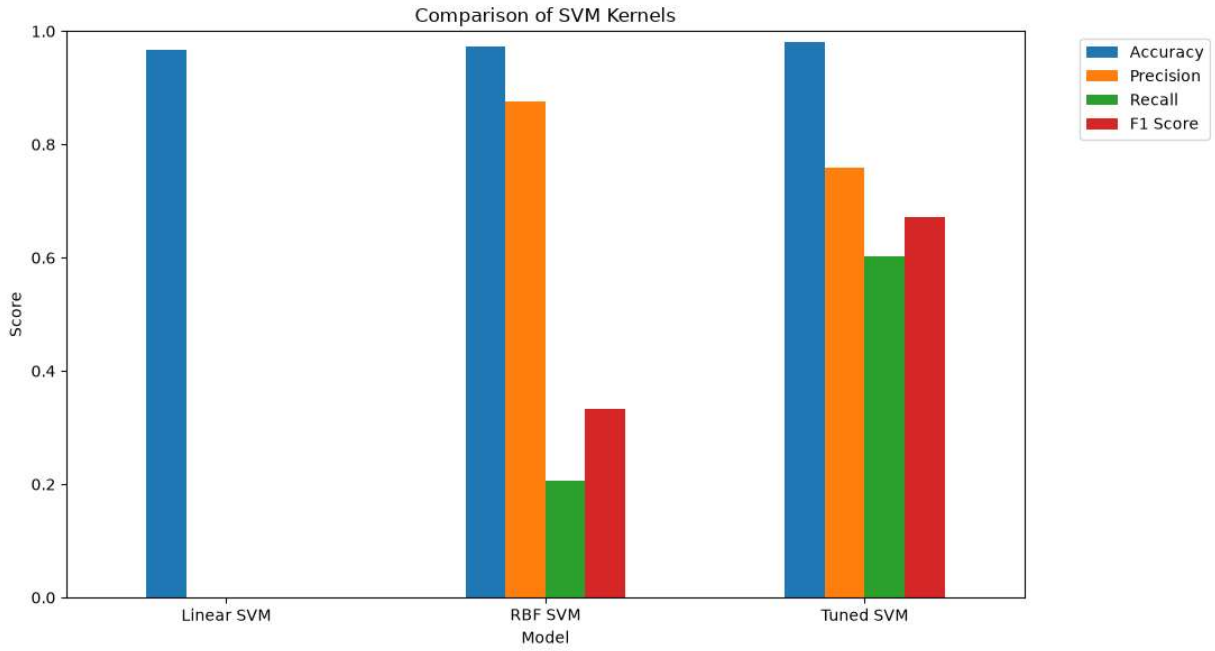
plt.title("Comparison of SVM Kernels")
plt.xlabel("Model")
plt.ylabel("Score")
```

```
plt.ylim(0, 1)

plt.xticks(rotation = 0)

plt.legend(bbox_to_anchor = (1.05, 1), loc = "upper left")

plt.tight_layout()
plt.show()
```



In [ ]: